

JSON Web Token (JWT)

1. What is JWT?

JSON Web Token (JWT) is an open standard (**RFC 7519**) that defines a **compact, self-contained, and secure mechanism** for transmitting information between parties as a JSON object.

JWTs are **digitally signed**, which guarantees:

- Integrity of the data
- Authenticity of the sender
- Trustworthiness of the payload

JWT can be signed using:

- **Secret key (HMAC – HS256)**
 - **Public/Private key pair (RSA, ECDSA)**
-

2. Why JWT is Used

2.1 Authorization (Primary Use Case)

JWT is most commonly used for **authorization**.

Flow:

1. User logs in with valid credentials
2. Server generates a JWT
3. Token is sent to client
4. Client sends token with every request
5. Server verifies token before serving protected APIs

JWT enables **stateless authentication**, meaning the server does not store session data.

2.2 Secure Information Exchange

JWT ensures:

- Data is digitally signed
 - Data cannot be altered without detection
 - Receiver can verify sender identity
-

3. JWT Structure

A JWT consists of **three parts**, separated by dots (.):

```
xxxxx.yyyyy.zzzzz
```

Parts:

1. Header
2. Payload
3. Signature

All parts are **Base64URL encoded**.

4. JWT Header

The header contains metadata about the token.

Example

```
{  
  "alg": "HS256",  
  "typ": "JWT"  
}
```

- **alg**: Signing algorithm
 - **typ**: Token type
-

5. JWT Payload

The payload contains **claims**.

Claims

Claims are statements about:

- The user
- Token metadata
- Application-specific data

Types of Claims

- **Registered:** `iss`, `exp`, `iat`, `sub`
- **Public:** email, role
- **Private:** uid, permissions

Example Payload

```
{
  "id": 101,
  "email": "user@gmail.com",
  "iat": 1712123456
}
```

⚠ Never store passwords or sensitive data in payload.

6. JWT Signature

The signature ensures the token is **not tampered**.

Signature Formula

```
HMACSHA256(
  base64UrlEncode(header) + "." + base64UrlEncode(payload),
  secret
)
```

- Uses algorithm from header
 - Uses secret key or private key
-

7. Final JWT Format

```
Base64URL(Header).Base64URL(Payload).Base64URL(Signature)
```

This compact format can be transmitted via:

- HTTP headers
 - Cookies
 - URLs
-

8. Secret Key Management

Secret Key File (`config.js`)

```
// Secret key used to sign and verify JWT
// Generated from https://randomkeygen.com/

const SECRET = 'kcbksdkfbwebweaibduewvufyiewbdiywefbuiibdsibd'

module.exports = { SECRET }
```

✓ Keep secret secure ✓ Never expose in frontend

9. Installing JWT Library

```
npm install jsonwebtoken
```

10. Creating (Signing) JWT

10.1 Importing

```
const jwt = require('jsonwebtoken')
const config = require('./config')
```

10.2 Preparing Payload

```
const payload = {
  id: user.id,
  email: user.email
}
```

10.3 Generating Token

```
const token = jwt.sign(payload, config.SECRET)
```

Token is returned to client after successful login.

11. Verifying JWT (Middleware)

11.1 Importing

```
const jwt = require('jsonwebtoken')
const config = require('./config')
```

11.2 Allowed Public URLs

```
const allowedUrls = ['/users/signup', '/users/signin']

if (allowedUrls.includes(req.url)) {
  next()
}
```

11.3 Token Extraction

```
const token = req.headers.token

if (!token) {
  res.send('Token is missing')
}
```

11.4 Token Verification

```
const payload = jwt.verify(token, config.SECRET)
```

If invalid:

- Error is thrown
 - Request is rejected
-

11.5 Attaching Payload to Request

```
req.headers.id = payload.id  
req.headers.email = payload.email  
next()
```

This allows protected APIs to identify the user.

12. JWT Authentication Flow

```
Client Login  
↓  
JWT Created  
↓  
Token Sent to Client  
↓  
Client Sends Token in Header  
↓  
Middleware Verifies Token  
↓  
Access Granted / Denied
```

13. Advantages of JWT

- Stateless authentication
 - No session storage required
 - Fast and scalable
 - Platform independent
-